

Contrôle continu - 3 exercices

L'exercice 3 doit être traité sur une copie séparée.

Exercice(s)

Exercice 1 – Horloges et Causalité

Question 1

On considère une exécution répartie sur 3 sites, pour laquelle on n'a pu récupérer que l'horloge vectorielle du dernier événement sur chacun des sites :

$\begin{bmatrix} 3 \\ 3 \\ 2 \end{bmatrix}$ sur le site 1, $\begin{bmatrix} 1 \\ 3 \\ 2 \end{bmatrix}$ sur le site 2, $\begin{bmatrix} 1 \\ 0 \\ 2 \end{bmatrix}$ sur le site 3.

1. Combien y a-t-il eu d'événements sur chacun des sites ?
2. Peut-on connaître le nombre de messages émis par le site 1 ?
3. Peut-on savoir si le site 2 a envoyé un message au site 1 ?

Chacune de vos réponses devra être clairement justifiée.

Question 2

On rappelle la propriété CO : $\forall m, m' \text{ deux messages, } send(m) \rightarrow send(m') \Rightarrow rec(m) \rightarrow rec(m')$

L'exécution de la figure 1 vérifie-t-elle la propriété CO ? Justifiez précisément votre réponse.

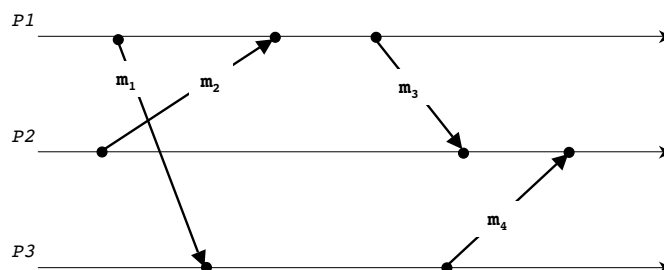


FIGURE 1 – Exécution répartie et causalité

Exercice 2 – Exécution de l'algorithme de Singhal et Kshemkalyani

Question 1

Complétez l'exécution de l'algorithme de Singhal et Kshemkalyani pour l'exemple de la figure 2. Vous préciserez les valeurs initiales des vecteurs LU et LS utilisés par l'algorithme ainsi que leurs valeurs *après* l'exécution de tout événement qui les modifie.

Vous donnerez aussi l'information véhiculée par les messages, en expliquant la manière dont elle est obtenue.

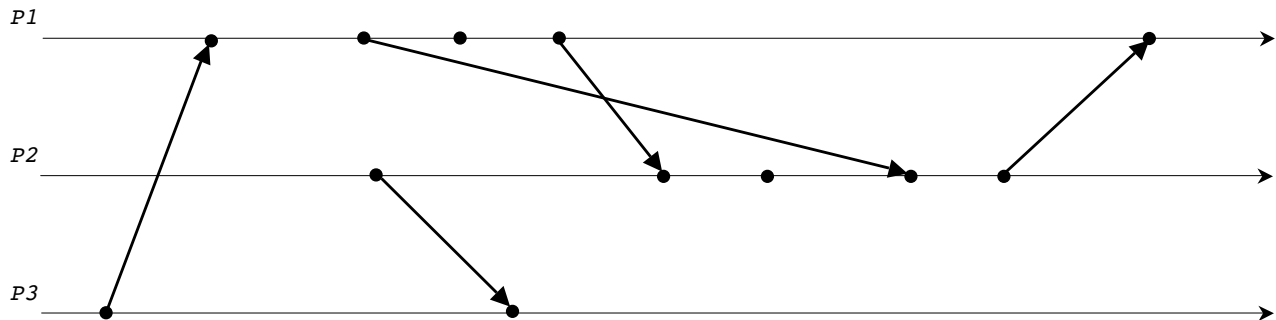


FIGURE 2 – Exécution de l'algorithme de Singhal et Kshemkalyani

Exercice 3 – Exclusion mutuelle

Nous considérons un réseau de N sites ayant une topologie en arbre. Chaque site connaît l'ensemble de ses voisins (qui ne change jamais), et il ne peut communiquer qu'avec eux. Contrairement à l'algorithme de *Naimi-Trehel*, ici un site ne connaît donc pas tous les autres sites.

Chaque site gère une variable *father*. Les pointeurs *father* (chemin des requêtes) sont orientés vers le site qui possède le jeton, qui se trouve donc à la racine de l'arbre.

La figure 3 montre une configuration initiale possible dans laquelle le site *A* possède le jeton.

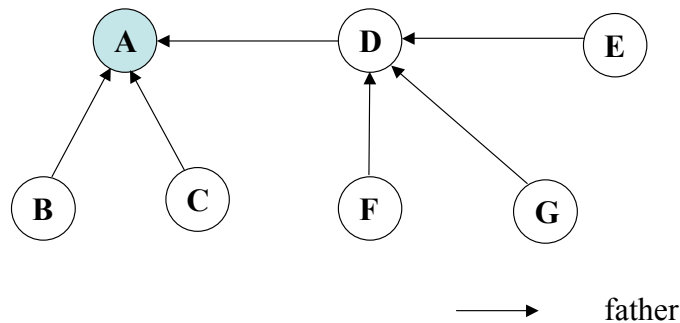


FIGURE 3 – Initialisation

Les requêtes d'entrée en section critique sont propagées vers la racine. Cependant, contrairement à l'algorithme de *Naimi-Tréhel* où S_i met à jour la variable $father_i$ lorsqu'il reçoit une requête de S_j ($father_i = S_j$), ici la variable $father_i$ est mise à jour à chaque fois que S_i envoie le jeton à son voisin S_j ($father_i = S_j$).

$father_i$ pointe donc toujours vers le dernier voisin auquel S_i a transmis le jeton. La figure 4 montre la configuration obtenue lorsque le site *G* reçoit le jeton suite à une demande : *A* a transmis le jeton à *D* ($father_A = D$), qui l'a transmis à *G* ($father_D = G$).

- *G* envoie sa requête à *D* qui la transmet à *A*.
- *A* envoie le jeton à *D* et change son *father*
- En recevant le jeton, *D* l'envoie à *G* et change son *father*

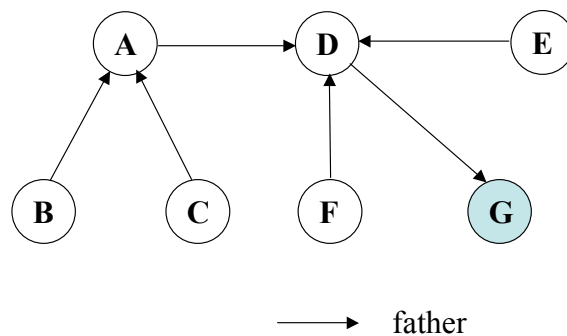


FIGURE 4 – *G* demande le jeton

Un site peut recevoir plusieurs requêtes d'entrée en section critique avant d'avoir reçu le jeton. Par conséquent, il va falloir que l'algorithme et les sites gèrent les requêtes pendantes, dans l'ordre : si un site reçoit deux requêtes il doit les **traiter dans l'ordre** y compris par rapport à sa propre requête, s'il en a une.

De plus, un site qui a déjà transmis une requête à son *father* et n'a pas encore reçu le jeton de celui-ci il ne transmet pas la deuxième requête tant qu'il n'a pas reçu le jeton.

Par exemple, dans la Figure 3, supposons que *F* et *G* veulent exécuter la section critique et que *D* reçoit dans l'ordre la requête de *G* puis celle de *F* avant d'avoir reçu le jeton de *A*. *D* transmet à *A* la requête de *G* uniquement, puis attend le jeton.

Question 1

Considérons l'exemple donné ci-dessus où *D* reçoit la requête de *G*, puis celle de *F* avant d'avoir reçu le jeton de

A. D doit-il, à un moment, transmettre la requête de F ? Si oui, à quel site et quand ? Sinon, pourquoi ? Donnez la configuration de l'arbre après que les deux requêtes ont été satisfaites.

Question 2

Un site peut-il recevoir le jeton, alors qu'il n'a pas demandé à rentrer en section critique ? Un site en attente pour exécuter la section critique et qui reçoit le jeton peut-il toujours entrer en section critique ? Justifiez vos réponses.

Question 3

Que proposez-vous comme structures de données pour qu'un site puisse assurer la vivacité de l'algorithme : contrôle de requêtes pendantes et transmission du jeton ?

Question 4

Donnez le pseudo-code de l'algorithme, en fournissant les trois fonctions :

- *Request_CS()* : demande de section critique faite par S_i .
- *Release_CS()* : libération de la section critique par S_i .
- *Receive_msg(type, S_j)* : appelée par S_i lors de la réception d'un message du type *REQ* (requête d'entrée en SC) ou du type *TOKEN* (jeton) envoyé par S_j . Remarquez que S_j est toujours l'identifiant d'un des voisins de S_i .

Vous pouvez ajouter les variables et fonctions que vous estimez nécessaires.

Question 5

Quelle est la complexité de l'algorithme en nombre de messages et dans le pire cas ? Quels sont les avantages et/ou inconvénients d'un tel algorithme par rapport à l'algorithme de *Naimi-Tréhel* ? Justifiez vos réponses.